

Actividad N°2 – Item I

Caso 1: NortiShop – Cuadro de protección de datos personales y financieros

Seguridad y Protección de Datos – Proyecto Integrador

Integrantes: Ignacia Castillo, Milan Kurte, Francisco Polo

Cuadro basado en el diccionario de datos de la Actividad N°1. Este cuadro se enfoca en los 12 datos personales y financieros con riesgo de confidencialidad (contraseñas, tarjetas, RUT, teléfonos, direcciones, correos y datos de comportamiento), que son los que se protegen con hashing, cifrado o tokenización. Los datos cuyo riesgo principal es de integridad o disponibilidad (stock, estado del pedido, reportes de ventas, etc.) quedan fuera de este cuadro. El teléfono del cliente registrado no estaba en el mapeo de la Semana 1: se incorporó al revisarlo y reemplaza la fila de ejemplo de la plantilla ("Número de tarjeta"), que ya queda cubierta por las filas de tarjeta guardada y tarjeta del invitado.

CAMPO	NECESIDAD	ELECCIÓN	CRITERIO TÉCNICO	IMPLEMENTACIÓN	EVIDENCIA
Teléfono del cliente registrado	Se necesita leer el número real para coordinar la entrega o contactar al cliente por su pedido	Cifrado (AES-256)	Tiene pocos valores posibles (9 dígitos), así que un hash se podría revertir probando todos los números; además hay que recuperarlo. Se cifra con AES-256-GCM, que también detecta si el valor fue alterado.	AES-256-GCM con contexto clientes.telefono:<fila> y llave versionada (v1:) leída del .env; se descifra con obtener_datos_cliente()	Item II, Figura 3: clientes.telefono_cifrado (v1:...). Figura 1: teléfono descifrado con obtener_datos_cliente() (demo.py, sección 3). Figura 5: el número no aparece en claro en la BD.
Correo y contraseña de la cuenta	La contraseña solo se necesita para comparar al iniciar sesión; el correo sí se necesita leer para enviar notificaciones	Hashing (contraseña) + Cifrado (correo)	bcrypt es lento a propósito (costo 12) y usa un salt único por usuario, lo que resiste fuerza bruta y tablas rainbow; por eso no se usa un SHA-256 simple. AES-256-GCM es cifrado autenticado (confidencialidad + detección de cambios). Como el correo cifrado no se puede buscar, se agrega un índice HMAC-SHA256 con una llave separada para el login.	bcrypt con salt (factor de costo 12) para la contraseña; AES-256-GCM para el correo, con contexto clientes.correo:<fila> y llave versionada (v1:) del .env; índice HMAC-SHA256 con llave separada para buscar en el login	Item II, Figura 3: tabla clientes con password_hash (\$2b\$12\$...), correo_cifrado y correo_indice. Figura 1: login aceptado con la clave correcta y rechazado con la incorrecta (demo.py, sección 2).
Tarjeta guardada para pagar más rápido	Permite comprar sin reingresar los datos, pero no se puede guardar la tarjeta completa	Tokenización	El token es un valor aleatorio sin relación matemática con la tarjeta y solo sirve para cobrar a través de la pasarela con las credenciales del comercio. Al no almacenar el PAN, NortiShop reduce su alcance frente a PCI DSS; el CVV no se guarda nunca. Para mostrar la tarjeta basta con la marca y los últimos 4 dígitos.	Se guarda el token asociado a la cuenta del cliente, entregado por la pasarela de pago	Item II, Figura 3: tabla tarjetas_guardadas (token_pasarela, ultimos4, marca). Figura 2: compra con tarjeta guardada aprobada usando solo el token (demo.py, sección 4). Figura 5: el número de la tarjeta no aparece en la BD.
Dirección de	Se necesita leer el valor real	Cifrado (AES-256)	AES-256-GCM (estándar NIST)	AES-256-GCM con contexto de	Item II, Figura 3:

CAMPO	NECESIDAD	ELECCIÓN	CRITERIO TÉCNICO	IMPLEMENTACIÓN	EVIDENCIA
despacho	para que el repartidor pueda despachar el pedido		con llave de 256 bits leída desde .env. El nonce aleatorio hace que dos direcciones iguales se vean distintas en la BD, y la etiqueta GCM detecta si el valor cifrado fue alterado.	la fila (clientes.direccion / pedidos.direccion) y llave versionada (v1:) leída del .env; se descifra con direccion_despacho()	clientes.direccion_cifrado y pedidos.direccion_cifrado (v1:...). Figura 2: dirección descifrada con direccion_despacho(); sección 7: dato alterado y dirección copiada a otro pedido → InvalidTag.
Historial de compras	El cliente y el administrador necesitan poder revisarlo después, no solo compararlo	Cifrado (AES-256)	El detalle de productos se guarda como JSON y se cifra completo con AES-256-GCM, así no queda a la vista qué compró cada cliente. El monto y la fecha quedan en claro porque se necesitan para la contabilidad y por sí solos no revelan los hábitos del cliente.	Cifrado a nivel de campo con AES-256-GCM sobre el detalle de cada pedido, con contexto pedidos.detalle:<código del pedido> y llave versionada (v1:)	Item II, Figura 3: pedidos.detalle_cifrado. Figura 2: historial descifrado con historial_compras() (demo.py, sección 5).
RUT y datos para la boleta/factura	Se necesita recuperar el valor real para emitir el documento tributario	Cifrado (AES-256)	El RUT tiene pocos valores posibles, por lo que un hash se podría revertir probando todos los RUT; además hay que recuperarlo. Por eso se cifra con AES-256-GCM y se descifra solo al emitir el documento tributario.	AES-256-GCM con contexto clientes.rut:<fila> y la llave versionada (v1:) del .env, la misma de los otros campos cifrados	Item II, Figura 3: clientes.rut_cifrado. Figura 1: RUT recuperado con obtener_datos_cliente() (demo.py, sección 3).
Correo y teléfono que deja para el envío (invitado)	Se necesita contactar al cliente invitado sin que quede expuesto si se filtra la BD	Cifrado (AES-256)	Mismo esquema AES-256-GCM, cada campo con su propio nonce. Al igual que el RUT, el teléfono tiene pocos valores posibles, por lo que un hash sería fácil de revertir probando todos los números.	AES-256-GCM aplicado a ambos campos, cada uno con su contexto (pedidos.invitado_correo / pedidos.invitado_telefono:<código del pedido>) y la llave versionada (v1:)	Item II, Figura 3: pedidos.invitado_correo_cifrado e invitado_telefono_cifrado (pedido 2). Figura 2: datos descifrados con datos_contacto_invitado() (demo.py, sección 5).
Datos de la tarjeta al pagar (invitado)	Igual que la tarjeta guardada, es un dato financiero que no debe almacenarse completo	Tokenización	La tarjeta solo pasa por la pasarela en el momento del pago; Nortishop recibe el token y nunca escribe el PAN ni el CVV en disco ni en logs.	Se usa el token de la transacción que entrega la pasarela, no se guarda el número	Item II, Figura 3: pedido 2 (es_invitado = 1) con token_pasarela y ultimos4 = 2222. Figura 5: el número de la tarjeta no aparece en la BD.
Clave para entrar al panel de administración	Solo se necesita comparar al momento del login del administrador, nunca leerla	Hashing	bcrypt con costo 12 y salt único. Es la credencial más crítica (da acceso a los datos de todos los clientes), así que se usa el mismo algoritmo lento; la comparación la hace bcrypt.checkpw().	bcrypt con salt (factor de costo 12)	Item II, Figura 4: tabla administradores con password_hash (\$2b\$12\$...). Figura 1: login del administrador aceptado y rechazado (demo.py, sección 2).
Monto y tarjeta	La tarjeta es información	Tokenización	La tarjeta se protege con el	Se almacena el token entregado	Item II, Figura 3: tabla pedidos

CAMPO	NECESIDAD	ELECCIÓN	CRITERIO TÉCNICO	IMPLEMENTACIÓN	EVIDENCIA
(tokenizada) de la compra	bancaria que pasa entre NortíShop y la pasarela y no debe guardarse en claro; el monto se necesita para la contabilidad		token. El monto queda en claro porque no identifica a la persona y se necesita para la contabilidad y la conciliación con la pasarela; junto a él se guarda el código de autorización.	por la pasarela (ej. Transbank/Stripe) junto al monto y al código de autorización; la respuesta solo se acepta si su firma HMAC es válida	con monto, token_pasarela, ultimos4 y autorizacion. Figura 2: compras aprobadas con su código de autorización; sección 7: pago falsificado o reutilizado → RECHAZADO.
Lista de correos del boletín	Marketing necesita poder leer y usar los correos para enviar las campañas	Cifrado (AES-256)	AES-256-GCM sobre cada correo. Para evitar suscripciones duplicadas sin descifrar la lista se usa el índice HMAC-SHA256 (correo_indice, UNIQUE), que no permite recuperar el correo.	AES-256-GCM sobre cada correo, con contexto boletin.correo:<índice> y llave versionada (v1:); descifrado solo al momento del envío	Item II, Figura 4: boletin_suscriptores con correo_cifrado y correo_indice. Figura 2: correos descifrados solo al enviar (correos_para_envio_boletin()), demo.py sección 6).
Datos de comportamiento de compra para campañas	Marketing necesita leer estos datos para segmentar a los clientes	Cifrado (AES-256)	Los datos se guardan como JSON cifrado con AES-256-GCM en un solo campo, así no se puede inferir ni consultar el perfil del cliente mirando la BD sin la llave.	Cifrado a nivel de campo (AES-256-GCM) sobre los datos de comportamiento de cada cliente, con contexto comportamiento.datos:<cliente> y llave versionada (v1:)	Item II, Figura 4: comportamiento_compra.datos_cifrado. Figura 2: datos descifrados con leer_comportamiento() (demo.py, sección 6).